Rapport technique : Puls-Events, assistant RAG

Puls-Events est une preuve de concept d'assistant qui recommande des événements culturels de Loire-Atlantique. Un RAG retrouve d'abord des fiches Open Agenda par le sens, puis Mistral rédige la réponse à partir de ces seules fiches. Sur 17 questions annotées, 13 trouvent un résultat pertinent ; le point faible mesuré est la recherche, pas la rédaction.

1. Livrables

Livrable demandé	Où le trouver	État
Système RAG fonctionnel	src/rag/, scripts/	Fait
API REST (FastAPI)	src/rag/api.py, /docs	Fait : /ask, /health, /rebuild
Rapport technique	Ce document, README.md, docs/journal.md	Fait
Tests unitaires	tests/ : 162 tests	Fait
Jeu de test annoté	eval/questions.json (20 questions), eval/annotations.json	Fait
Dépôt GitHub	BENNEKROUF-AI-Dev/puls-events-rag	Fait : version finale

2. Architecture

Deux phases : un index construit une fois, puis une chaîne exécutée à chaque question.

HORS LIGNE (une fois)

Open Agenda --> nettoyage --> découpage --> embeddings --> index Faiss
15 925 fiches 12 730 événements 14 815 morceaux 384 dimensions 14 815 vecteurs

EN LIGNE (chaque question)

Question --> même modèle d'embeddings --> 5 morceaux les plus proches

--> Mistral rédige à partir de ces fiches --> réponse + sources

Le modèle d'embeddings doit être le même des deux côtés, sinon fiches et questions ne sont pas comparables. Aucune règle métier ne vit dans api.py : il traduit du HTTP vers chain.py et inversement.

3. Choix technologiques

Brique	Choix	Raison
Source	Open Agenda, Loire-Atlantique	Le Nord comptait 42 % de fiches d'un agenda non culturel
Embeddings	paraphrase-multilingual-MiniLM-L12-v2 via fastembed, local	Français, gratuit, sans limite d'appels (Mistral renvoyait des erreurs 429)
Index	Faiss IndexFlatIP, vecteurs normalisés	Cosinus exact ; à 15 000 vecteurs la recherche exhaustive prend quelques millisecondes
Génération	Mistral ministral-3b-latest, température 0	Reformuler des fiches, pas raisonner ; le seul appel externe
Orchestration	LangChain	Chaîne et prompt
API	FastAPI, Pydantic	Documentation générée, validation, injection de dépendances
Conteneur	Docker en deux étapes, utilisateur non administrateur	Reproductible ; clé d'API passée à l'exécution, jamais dans l'image

Les versions des bibliothèques sont figées dans requirements.txt : Ragas ne fonctionne pas avec langchain-community 0.4 ni avec le SDK mistralai 2.x.

4. Données et découpage

15 925 fiches collectées, 12 730 après nettoyage (hors zone, dates incohérentes, annulées, doublons). Seuls 2 207 événements sont encore à venir, soit 17 % : un filtre sur la date de fin écarte les événements terminés.

Les descriptions sont découpées en morceaux de 1 200 caractères avec 150 de recouvrement. Une fiche courte reste entière. Titre, lieu et date sont recopiés en tête de chaque morceau pour qu'il reste rattaché à son événement. Le résultat est de 14 815 morceaux, soit environ 1,16 par événement.

5. API

Route	Rôle	Comportement
POST /ask	Question, réponse et sources	Validée par Pydantic ; si Mistral est indisponible, renvoie les 5 événements trouvés avec llm_disponible: false
GET /health	État du service	200 si l'index est présent, 503 sinon, avec le détail
POST /rebuild	Reconstruire l'index	Réponse 202, tâche de fond, jeton obligatoire (403 sans jeton) ; le nouvel index est construit à côté puis installé par renommage de dossier

6. Tests et industrialisation

162 tests automatisés sur 10 fichiers, dont la plupart tournent sans Internet, sans clé et sans modèle : l'API Open Agenda, les embeddings et le LLM sont simulés. La consigne donnée au modèle est testée comme du code. scripts/api_test.py vérifie un vrai serveur sur le vrai index : 23 vérifications sur 23.

Dans l'image Docker, la dernière mesure donnait 149 tests réussis et 10 ignorés (les tests de qualité des données, exclus par .dockerignore), à un moment où la suite comptait 159 tests. Trois tests ont été ajoutés depuis. Le nombre exact dans l'image reste à remesurer après reconstruction.

7. Évaluation et résultats

Protocole. Deux mesures séparées, car le système peut échouer en deux endroits : la recherche rate le bon événement, ou la rédaction s'écarte des fiches. 20 questions écrites comme les poserait un habitant, dont 3 hors périmètre comptées à part. Pour les 17 autres, annotation manuelle des 5 premiers résultats, puis Ragas avec un second modèle comme juge.

Mesure	Valeur	Ce qu'elle dit
Précision@5	0,447	Moins d'un résultat sur deux est pertinent
MRR	0,615	Le premier bon résultat arrive entre la 1re et la 2e place
Taux de réussite	0,765	13 questions sur 17 ont au moins un bon résultat
Fidélité (Ragas)	0,936	Les réponses évaluées restent largement fidèles aux fiches retrouvées
Pertinence de la réponse (Ragas)	0,500	La réponse répond à la question une fois sur deux
Précision du contexte (Ragas)	0,619	Instable : de 0,373 à 0,647 selon l'exécution

Interprétation. L'annotation manuelle (13 sur 17) et le juge automatique (0,5) n'ont rien en commun et concluent pareil : le problème est la recherche. Une réponse fidèle à de mauvaises fiches reste une mauvaise réponse : la fidélité n'exclut pas toute hallucination et ne garantit pas que le bon événement ait été retrouvé.

Limites de la mesure. 17 questions : ce sont des ordres de grandeur. Le juge est un petit modèle instable, et c'est le même qui rédige. Le rappel a été retiré : il valait 0,765, exactement le taux de réussite, car seuls les 5 résultats affichés sont annotés.

Dépendance au calendrier. Les annotations désignent des événements précis, par identifiant, et la recherche écarte les événements terminés. Rejouer l'évaluation à une autre date donne donc d'autres chiffres : au 20 octobre 2026, la précision@5 tombe à 0,235, le MRR à 0,471 et le taux de réussite à 0,471, parce que les événements annotés comme pertinents le 2 octobre ne sont plus proposés. Six fiches annotées, liées à q03, q08, q10, q11 et q15, ont été vérifiées : elles se sont terminées entre le 3 et le 18 octobre. Ce n'est pas une dégradation de la recherche : le système refuse de recommander un événement passé. Les chiffres de ce rapport sont valables à la date d'annotation. De plus, un événement correct mais absent des annotations est compté comme faux. Pour une évaluation reproductible, il faudrait figer la date de référence ou ré-annoter.

8. Défauts observés

Famille	Exemple	Cause
Le temps	« ce week-end » renvoie décembre	Un embedding ne porte pas de date
Le lieu	« du côté de Saint-Nazaire » renvoie Saint-Herblain, à 60 km	Noms proches, aucune notion de distance
La forme	« un festival » renvoie cinq concerts isolés	Le type d'événement n'est pas encodé
La diversité	Cinq résultats pour deux événements	Variantes d'un titre non dédoublonnées

Le prix ne peut pas être traité : le champ `conditions` est vide pour une grande partie des fiches et mélange tarif, inscription et jauge. C'est une limite de la source.

9. Incidents

- **Erreur 429 pendant deux jours :** la configuration pointait vers un modèle plafonné ; un second 429 signifiait serveurs saturés. `/health` affiche désormais le modèle chargé.
- **Certificat refusé :** l'antivirus inspectait le trafic HTTPS. Certificats racine ajoutés au magasin de l'environnement.
- **Ragas renvoyait NaN :** deux bogues de la bibliothèque, corrigés par un adaptateur.
- **Sonde de santé trompeuse :** `/health` renvoyait 200 sans index, donc Docker annonçait prêt un conteneur muet. Corrigé par un 503 et trois tests.

10. Pistes d'amélioration

Ces pistes sont des hypothèses : aucune n'a été testée. Seul le filtre sur la date existe aujourd'hui.

1. Filtrer sur l'heure, le jour et la ville, puis rejouer les 20 questions pour mesurer le gain.
2. Dédoublonner les variantes de titre et forcer la diversité des résultats.
3. Juger avec un modèle plus puissant que celui qui rédige.
4. Ajouter une recherche hybride (sens et mots-clés) pour retrouver le type d'événement.
5. Rejouer les tests à chaque publication.

11. Reproduire

```
git clone https://github.com/BENNEKROUF-AI-Dev/puls-events-rag.git
cd puls-events-rag
python -m venv venv
pip install -r requirements.txt
copy .env.example .env        # puis renseigner MISTRAL_API_KEY
python scripts/fetch_data.py
python scripts/build_index.py  # environ 22 minutes
pytest
docker compose up --build
```

Puis http://127.0.0.1:8000/docs. L'index est monté depuis ./data, la clé est lue dans .env au démarrage.

BENNEKROUF ADAM.

Le 10 Octobre 2026